



NPTEL Course on “Computer System Optimization for ML”

Lecture #12



Agenda

- **Recap**
- **Dataflow for DNN Execution (contd)**

NPTREL

1-D Convolution – Output Stationary



```
int I[H]; // Input activations
int W[R]; // Filter weights
int O[E]; // Output activations
```

```
for (e = 0; e < E; e++)
  for (r = 0; r < R; r++)
    O[e] += I[e+r] * W[r];
```

No constraints
on loop
permutations!

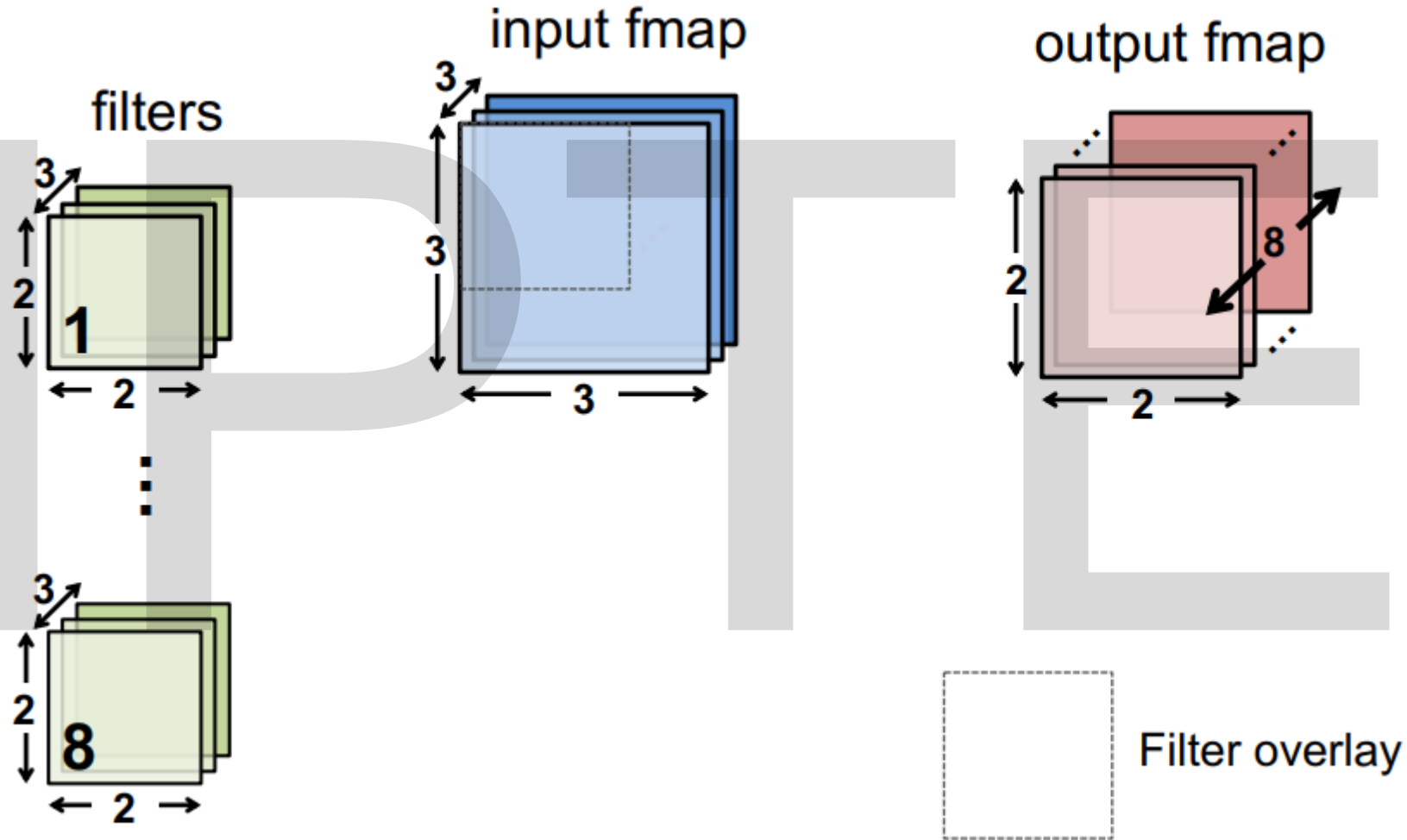
- **For the above loop, we essentially duplicate the weights on all PEs (outer loop)**
 - After duplication, each PE can execute the inner loop
 - The input activations (I) in one window (red box) go over all PEs, then the execution moves on to the next window
 - When all windows are completed, each PE gets the outputs locally

Agenda

- Recap
- **Dataflow for DNN Execution (contd)**

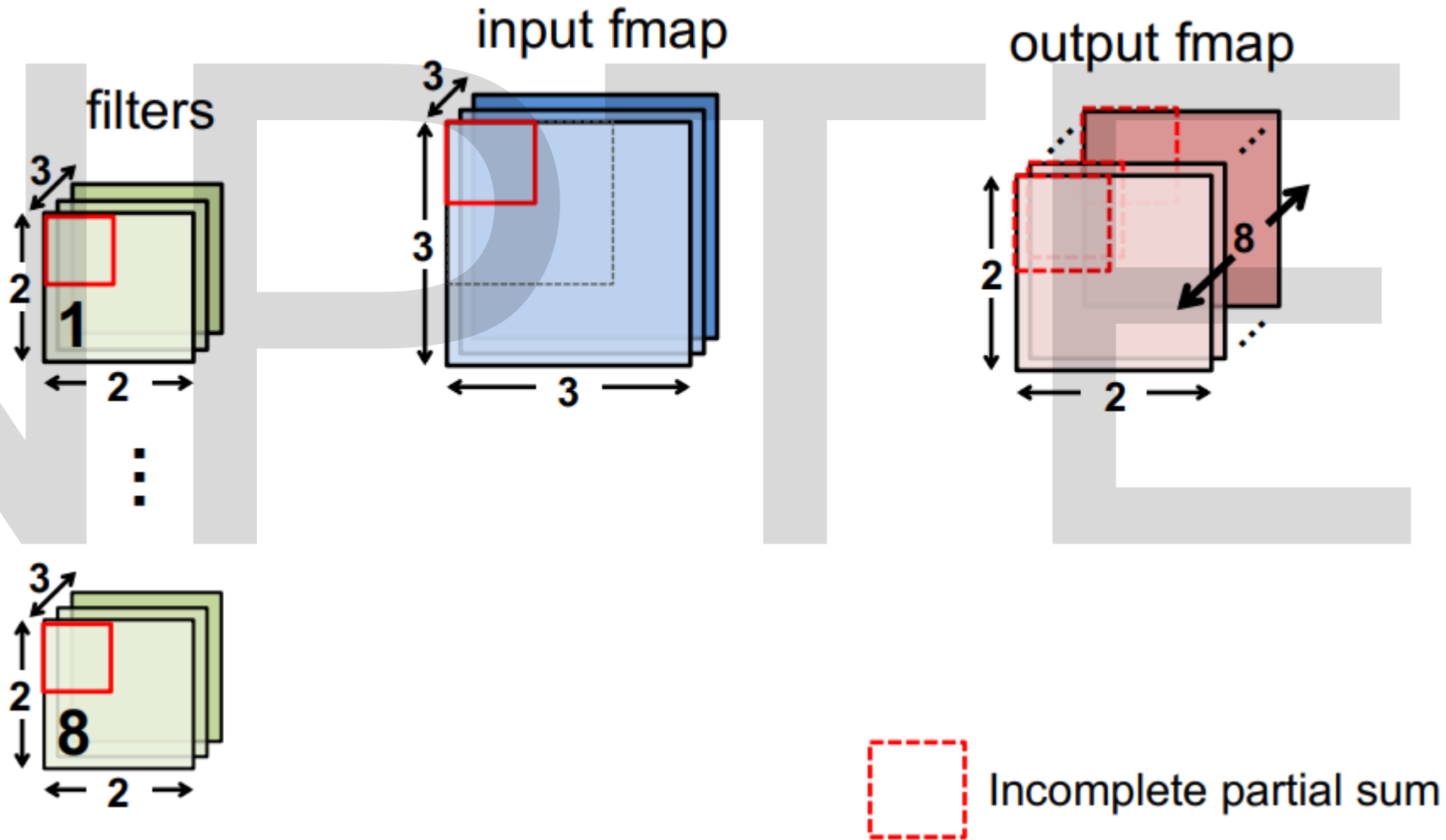
NPTEL

2-D Convolution – Output Stationary



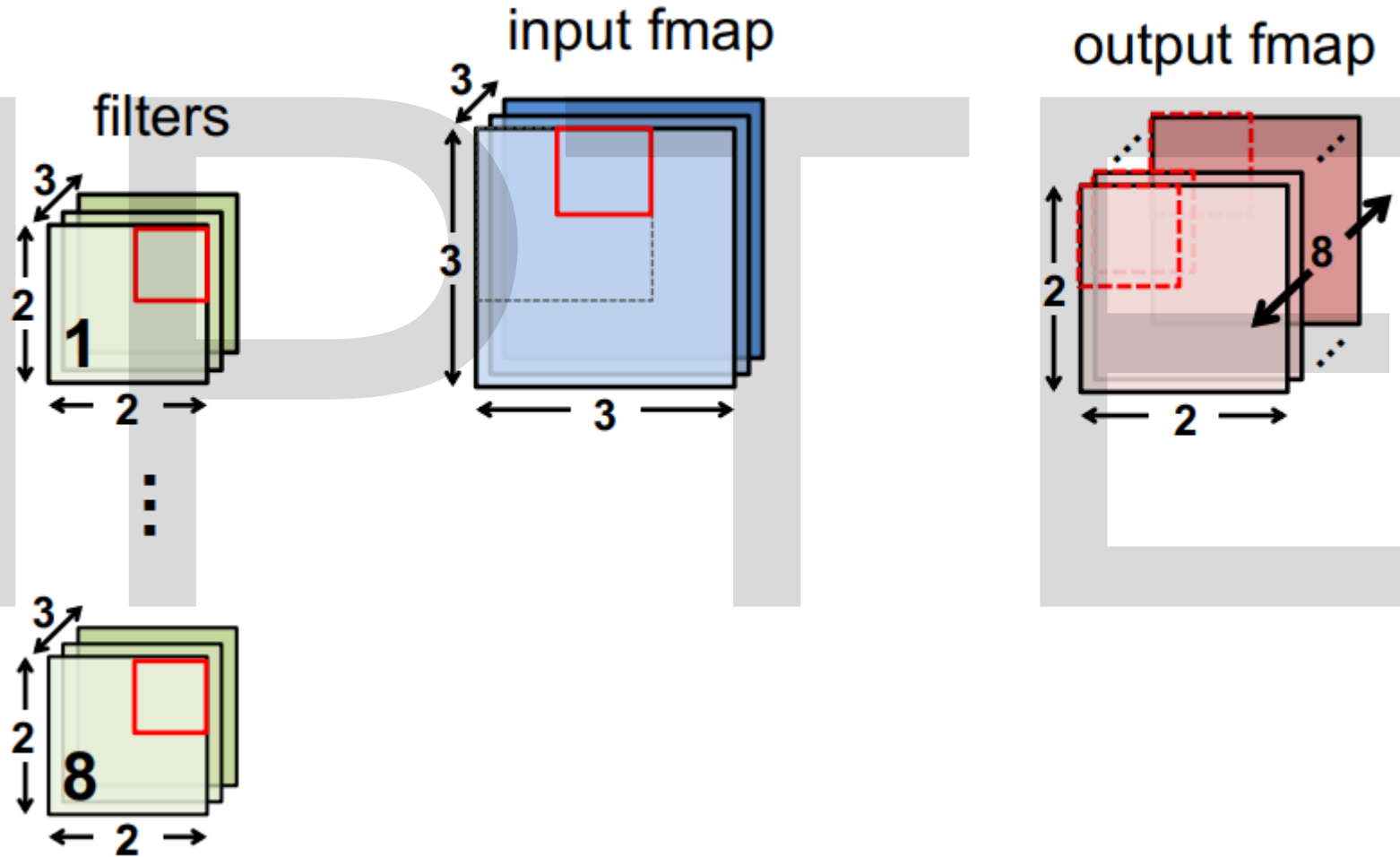
2-D Convolution – Output Stationary (1)

Cycle through input fmap and weights (hold psum of output fmap)



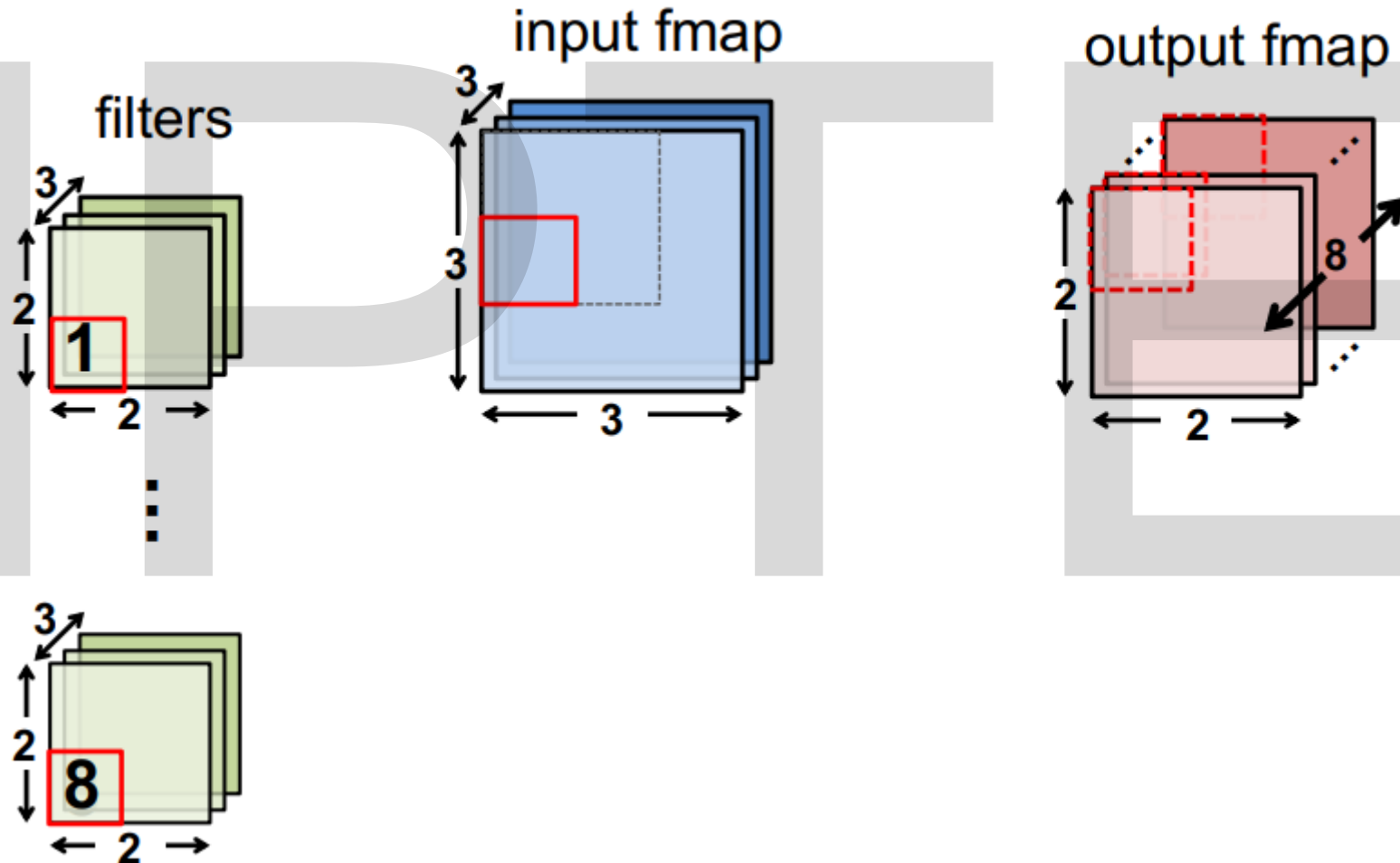
2-D Convolution – Output Stationary (2)

Cycle through input fmap and weights (hold psum of output fmap)



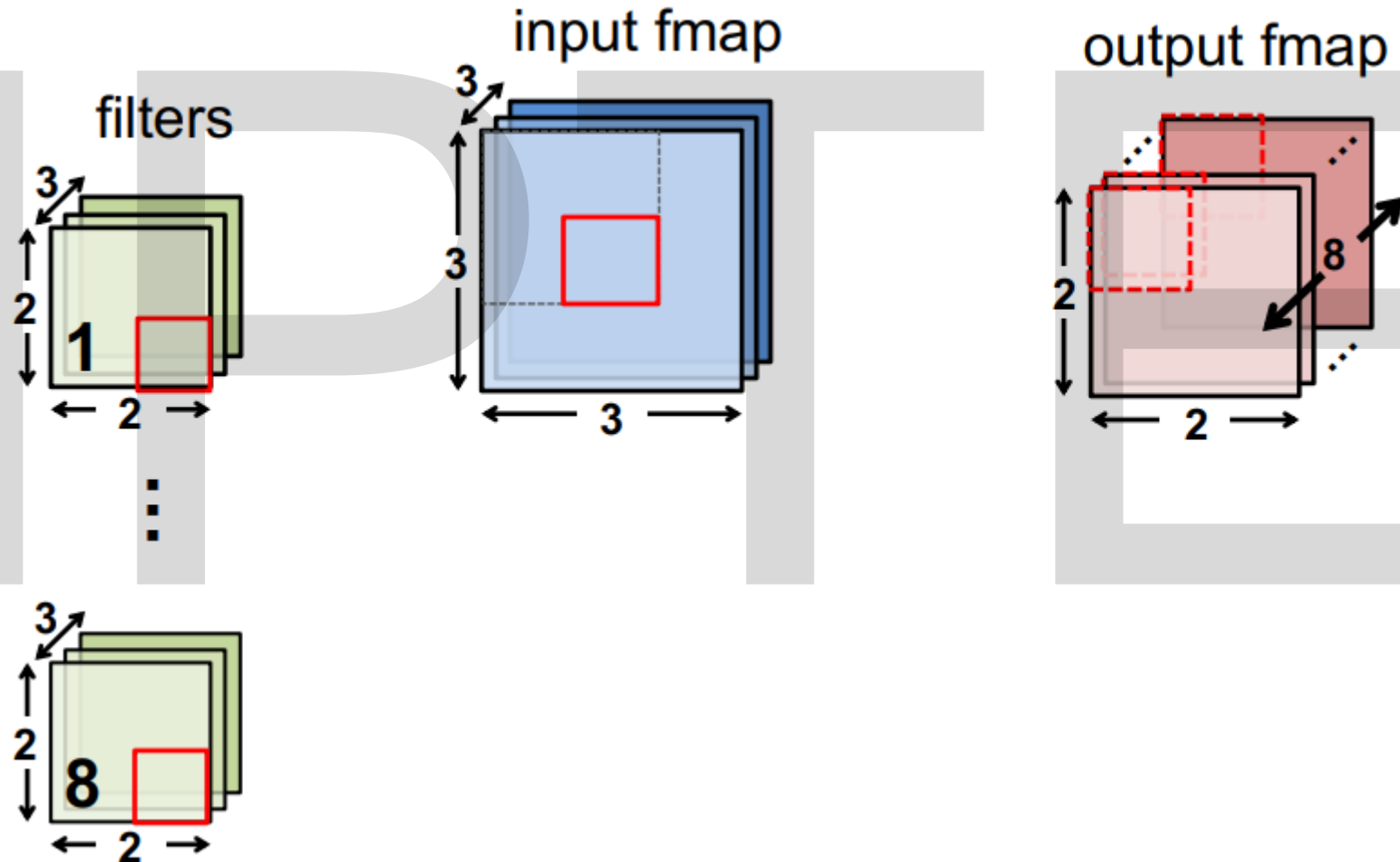
2-D Convolution – Output Stationary (3)

Cycle through input fmap and weights (hold psum of output fmap)



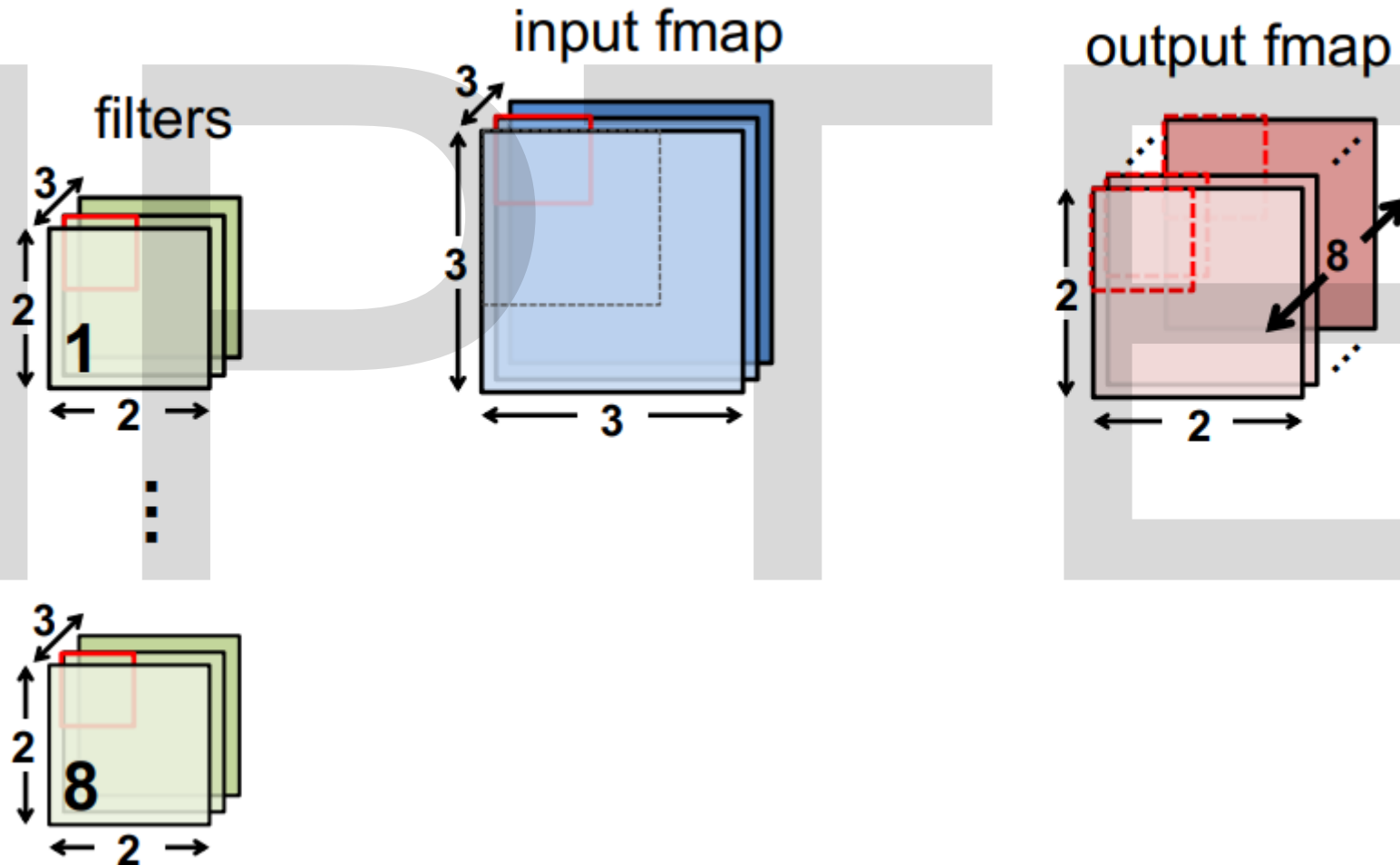
2-D Convolution – Output Stationary (4)

Cycle through input fmap and weights (hold psum of output fmap)



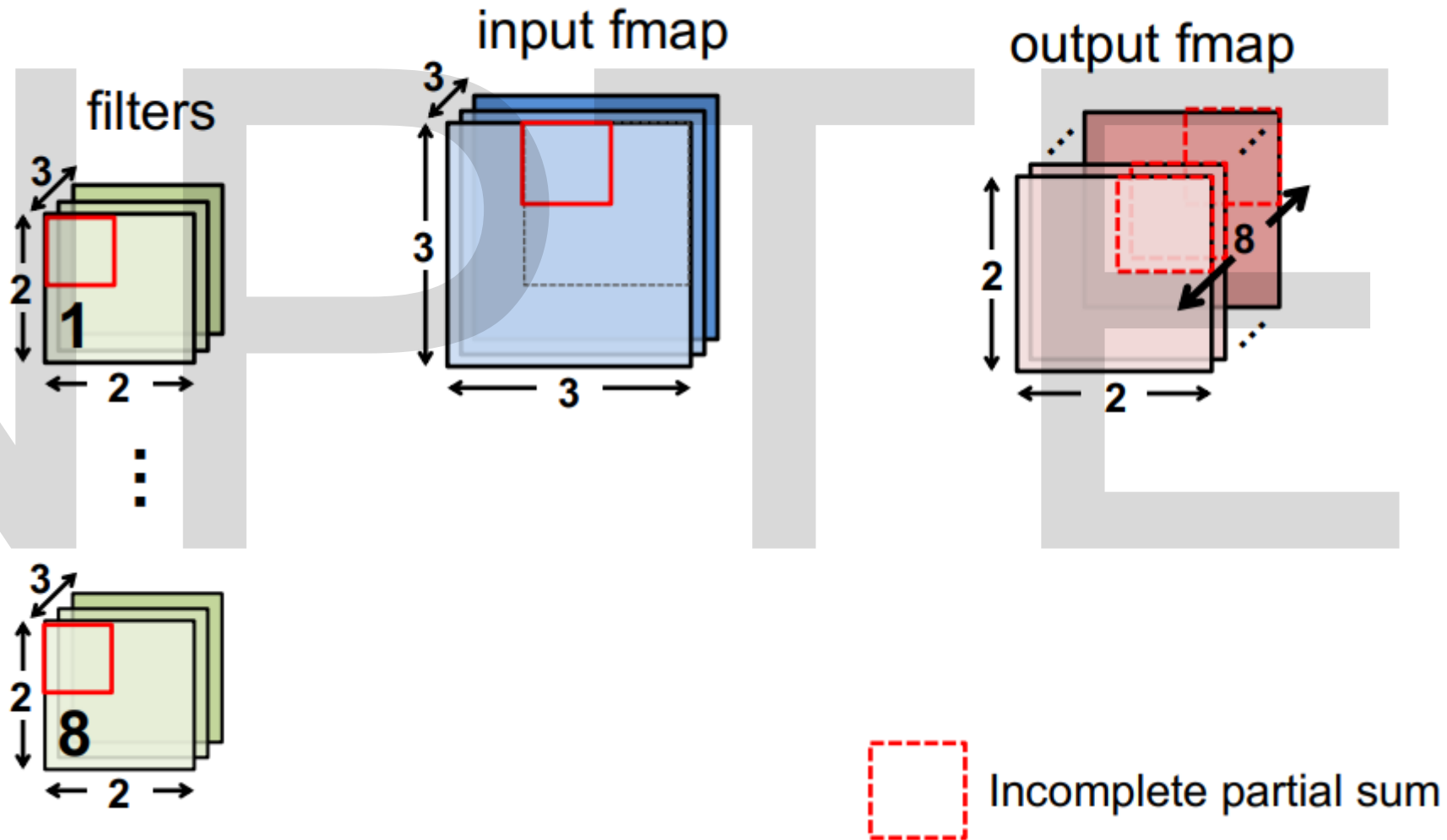
2-D Convolution – Output Stationary (5)

Cycle through input fmap and weights (hold psum of output fmap)



2-D Convolution – Output Stationary (6)

Cycle through input fmap and weights (hold psum of output fmap)

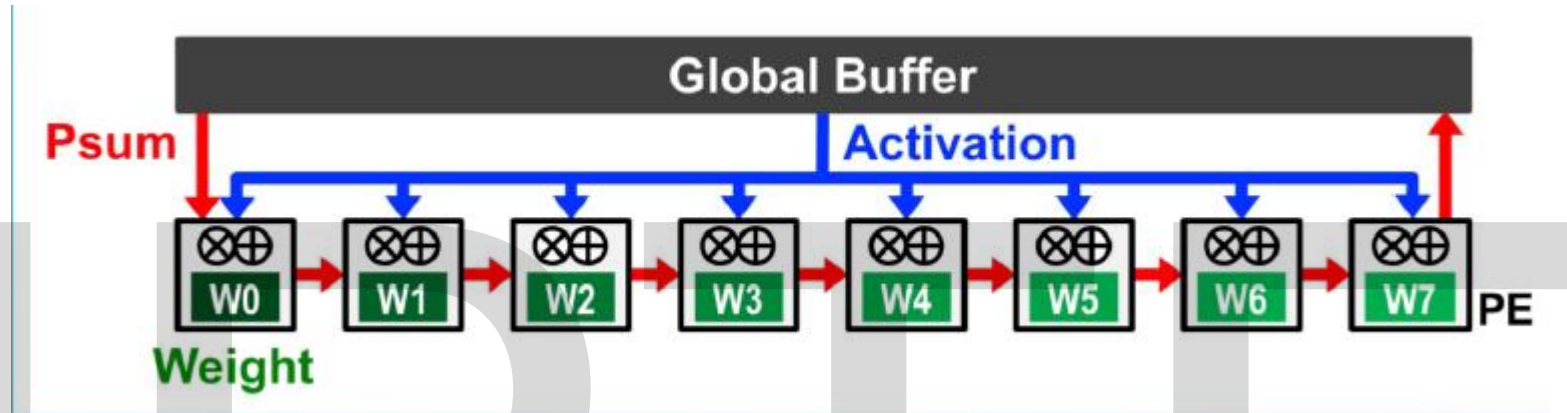


Different Types of Data Flow

- Output Stationary (OS)
- **Weight Stationary (WS)**
- Input Stationary (IS)

NPTEL

Weight Stationary (WS) Dataflow



- **Broadcast activations and accumulate partial sums (psums) spatially across the PE array**
- **Weights are stored in PE array**
 - Minimize read energy consumption
 - Maximize convolutional and filter reuse of weights
- **Weights Stationary**
 - Weights do not move (stay in PEs) until all computations related to them are finished

1-D Convolution – Weight Stationary



```
int I[H];    // Input activations
int W[R];    // Filter weights
int O[E];    // Output activations

for (r = 0; r < R; r++)
    for (e = 0; e < E; e++)
        O[e] += I[e+r] * W[r];
```

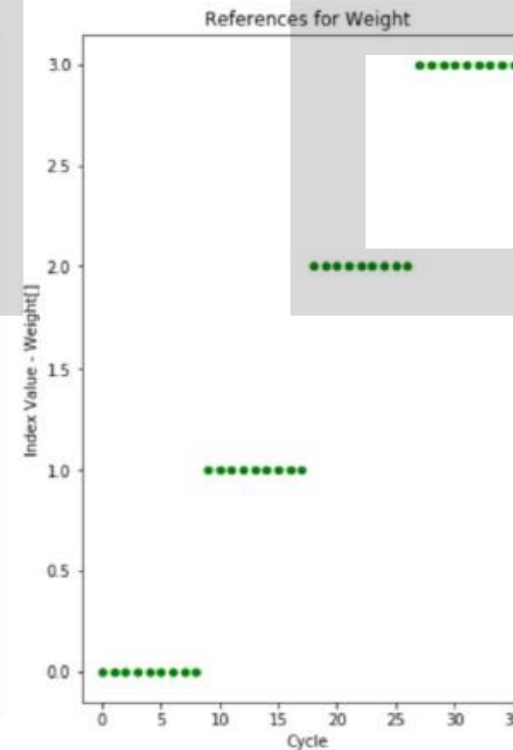
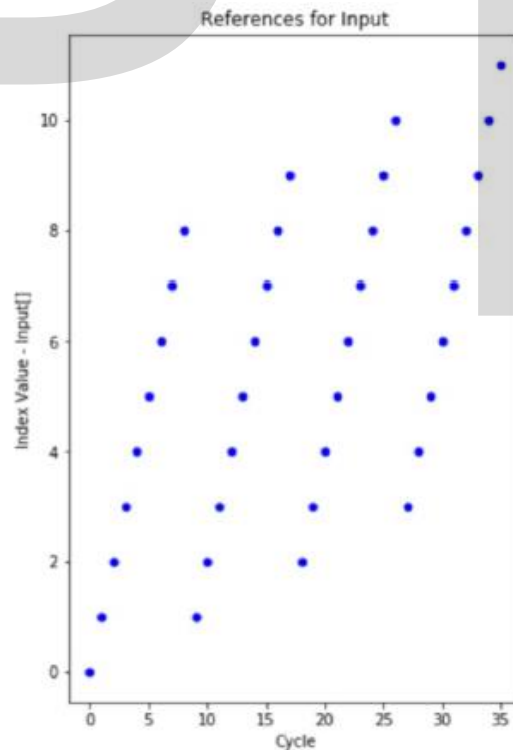
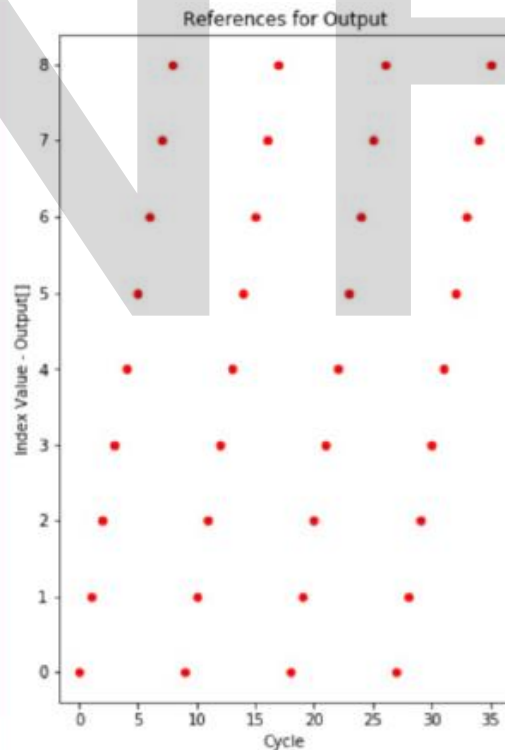
Weight Stationary – Reference Pattern

■ Observations

- Single **weight** is reused many times (E)
- Large sliding window of **inputs** (E)
- Fixed window of **outputs** (E)

```
int I[H];    // Input activations
int W[R];    // Filter weights
int O[E];    // Output activations
```

```
for (r = 0; r < R; r++)
    for (e = 0; e < E; e++)
        O[e] += I[e+r] * W[r];
```



Layer Shape:

- H = 12
- R = 4
- E = 9

Data Access for WS

Output Stationary

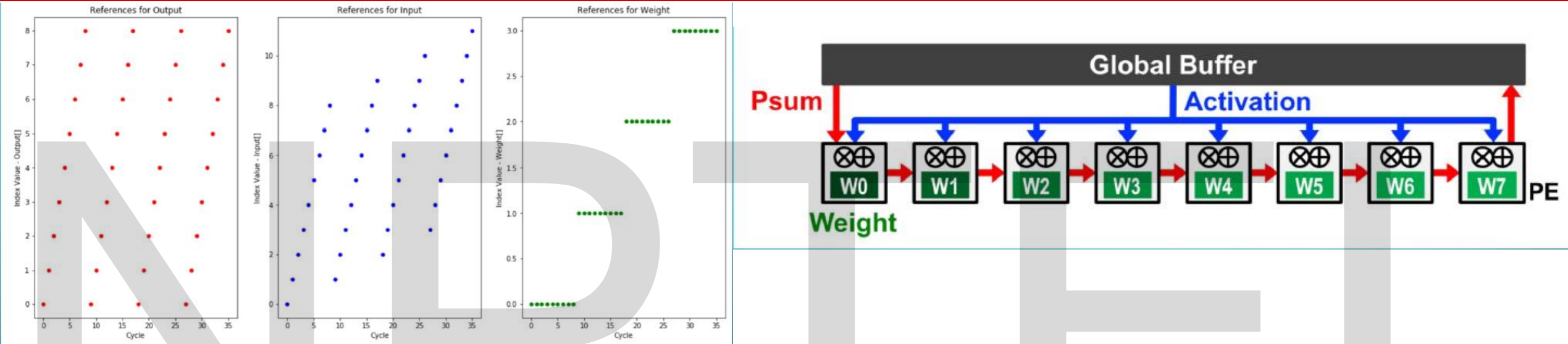
```
for (e = 0; e < E; e++)  
  for (r = 0; r < R; r++)  
    O[e] += I[e+r] * W[r];
```

Weight Stationary

```
for (r = 0; r < R; r++)  
  for (e = 0; e < E; e++)  
    O[e] += I[e+r] * W[r];
```

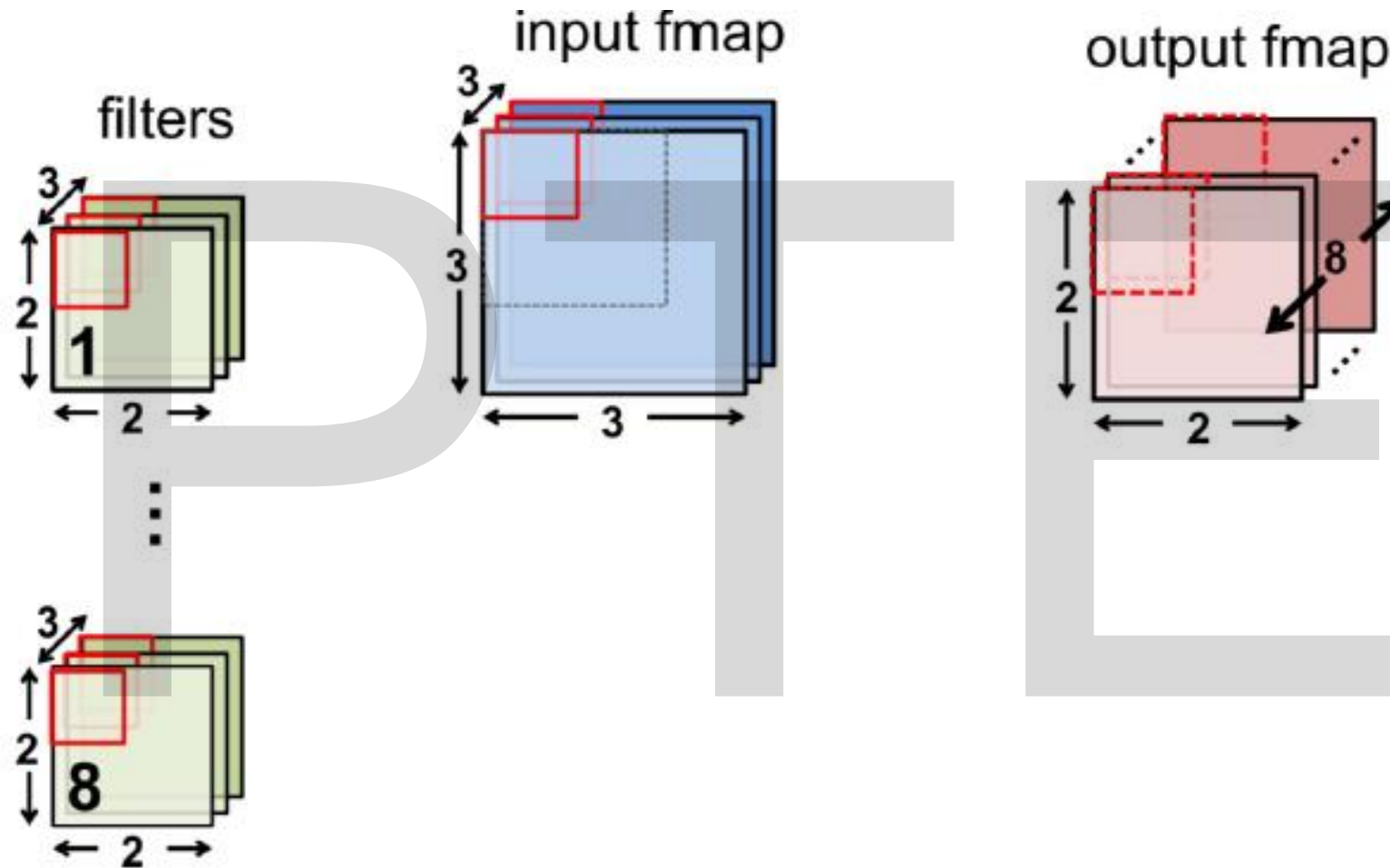
	OS	WS		
MACs	$E \cdot R$	$E \cdot R$		
Weight Reads	$E \cdot R$	R		
Input Reads	$E \cdot R$	$E \cdot R$		
Output Reads	0	$E \cdot R$		
Output Writes	E	$E \cdot R$		

Relation to WS Architecture

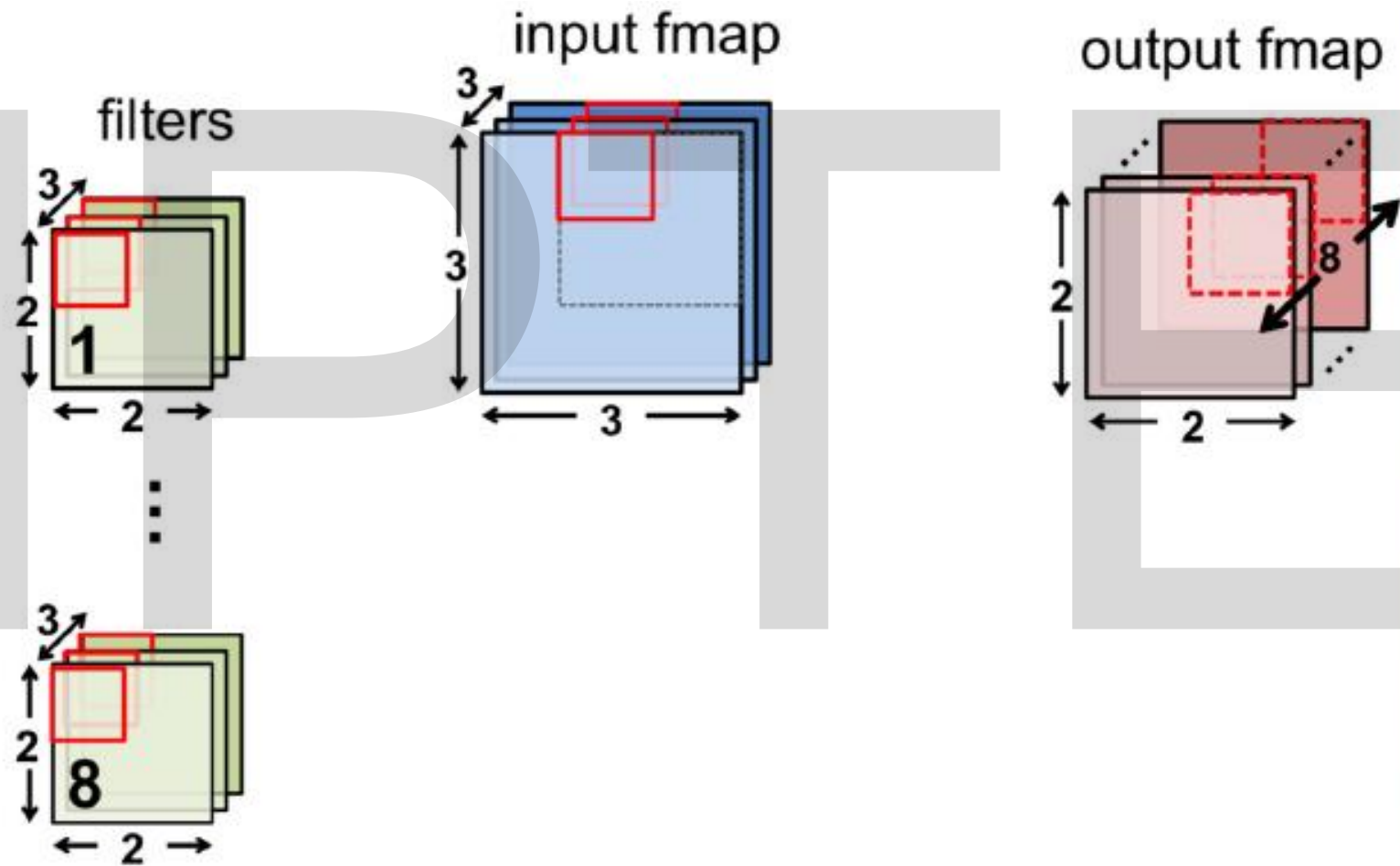


- In WS architecture, weights stay in each PE – P0:W0, P1:W1...
- Inputs are broadcasted into all PEs: $P_i: I[0], I[1], I[2], \dots$
- During computation, each PE performs the partial sum at each cycle and sends to the next PE – spatially accumulate
 - $O[0] = I[0]W[0] + I[1]W[1] + I[2]W[2] + I[3]W[3]$
 - $O[6] = I[6]W[0] + I[7]W[1] + I[8]W[2] + I[9]W[3]$
- P0 -> P1 -> P2 -> P3

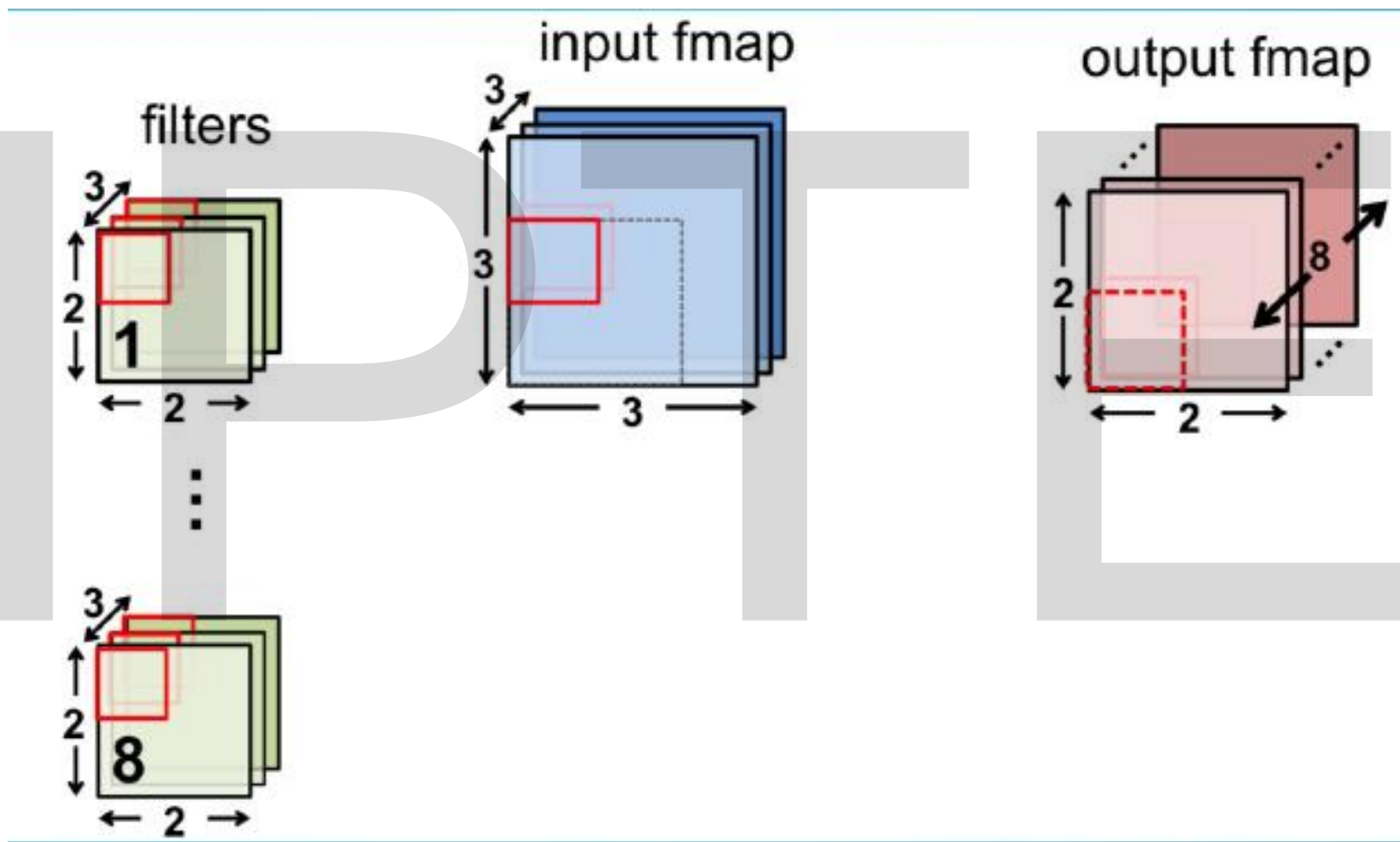
2-D Convolution – Weight Stationary (1)



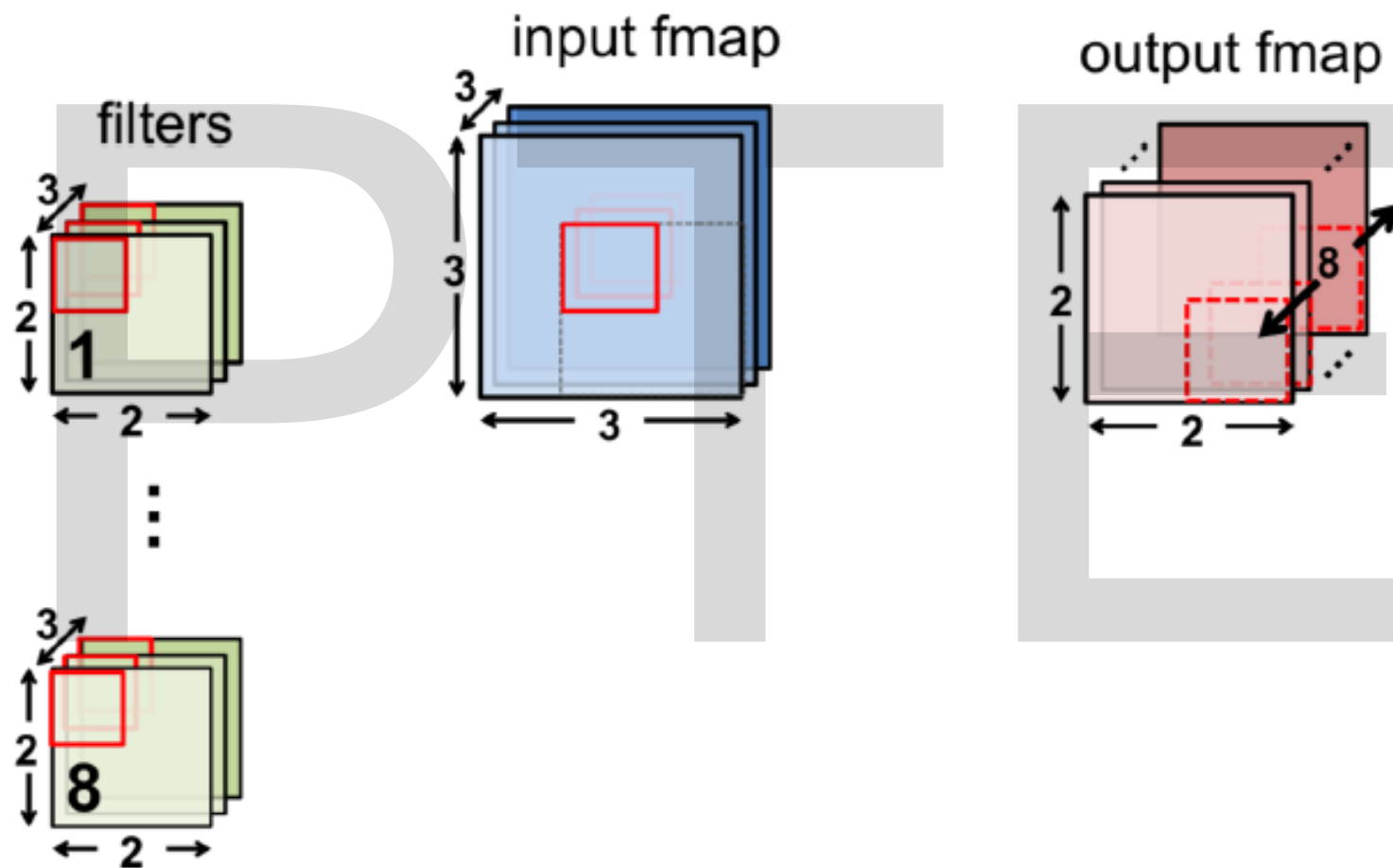
2-D Convolution – Weight Stationary (2)



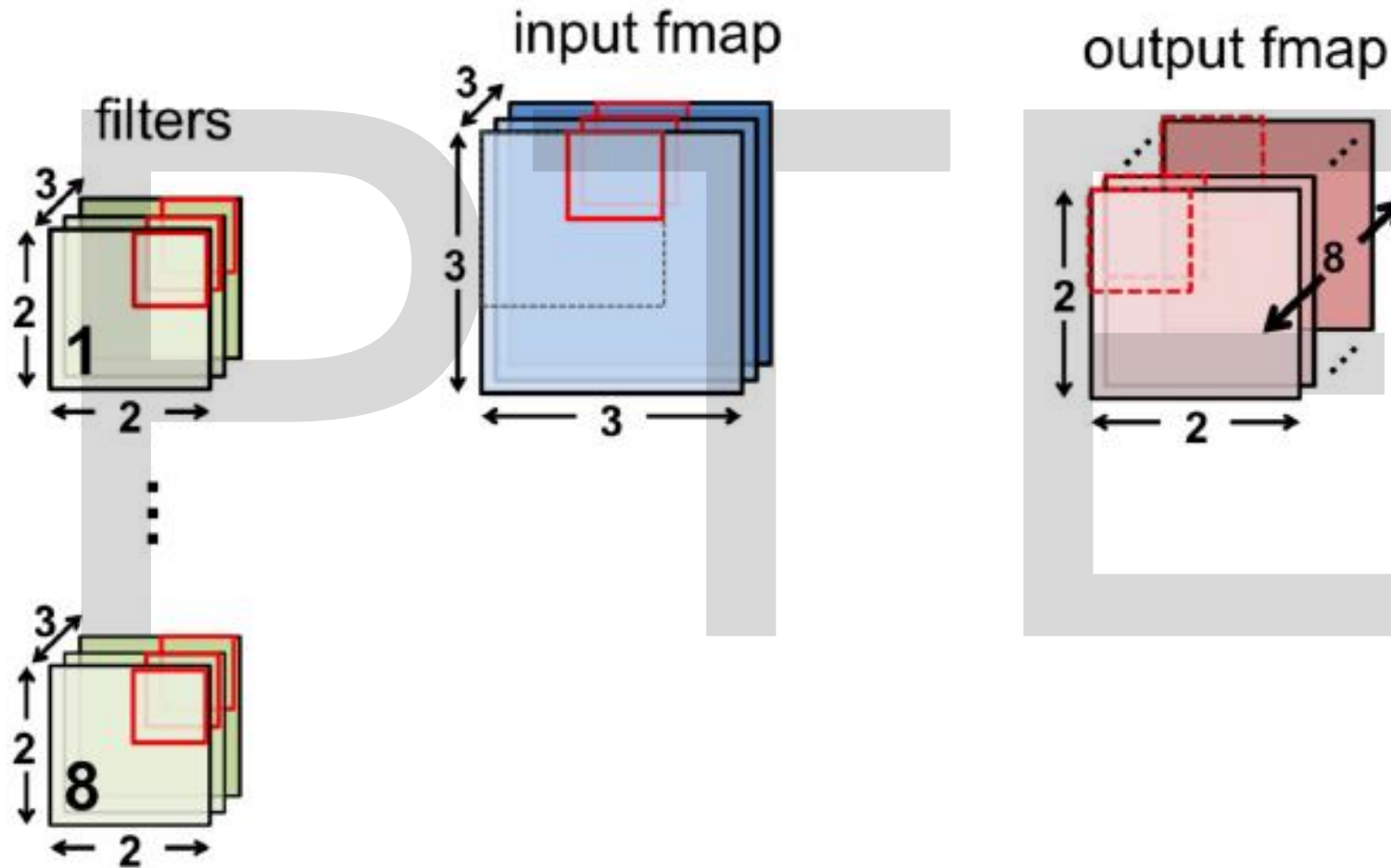
2-D Convolution – Weight Stationary (3)



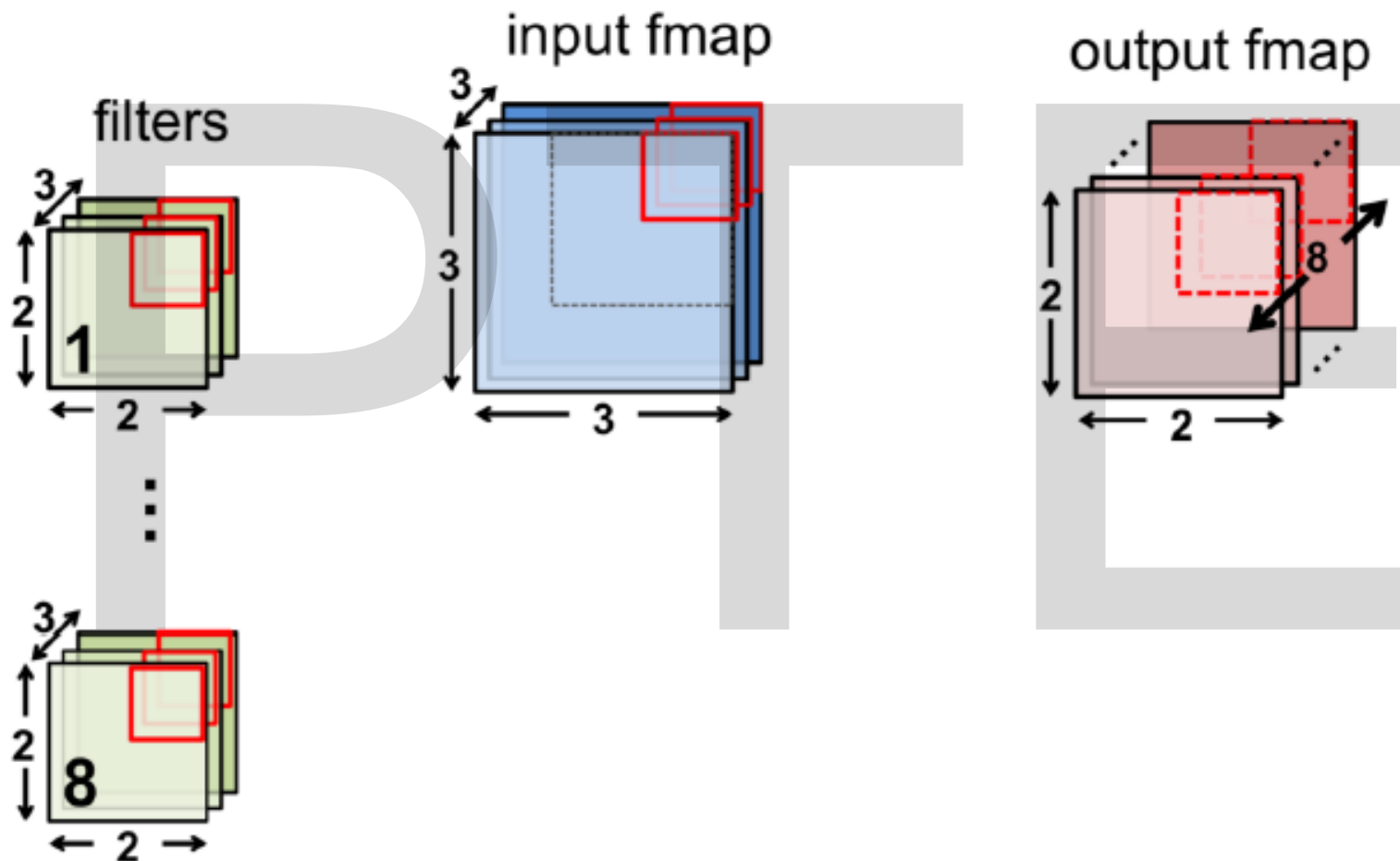
2-D Convolution – Weight Stationary (4)



2-D Convolution – Weight Stationary (5)



2-D Convolution – Weight Stationary (6)



WS Example: NVDLA (simplified)

- NVIDIA Deep Learning Accelerator (NVDLA)

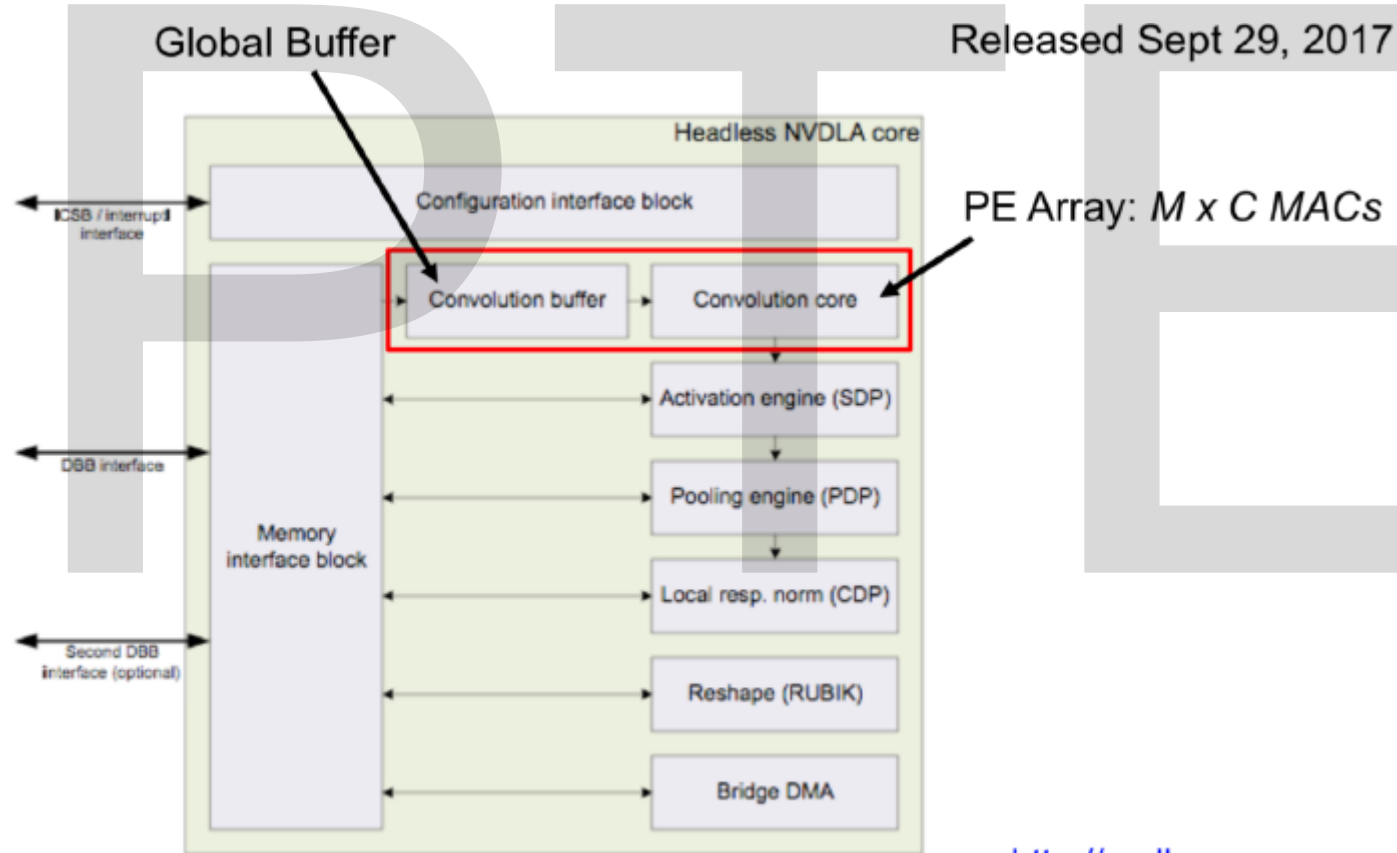
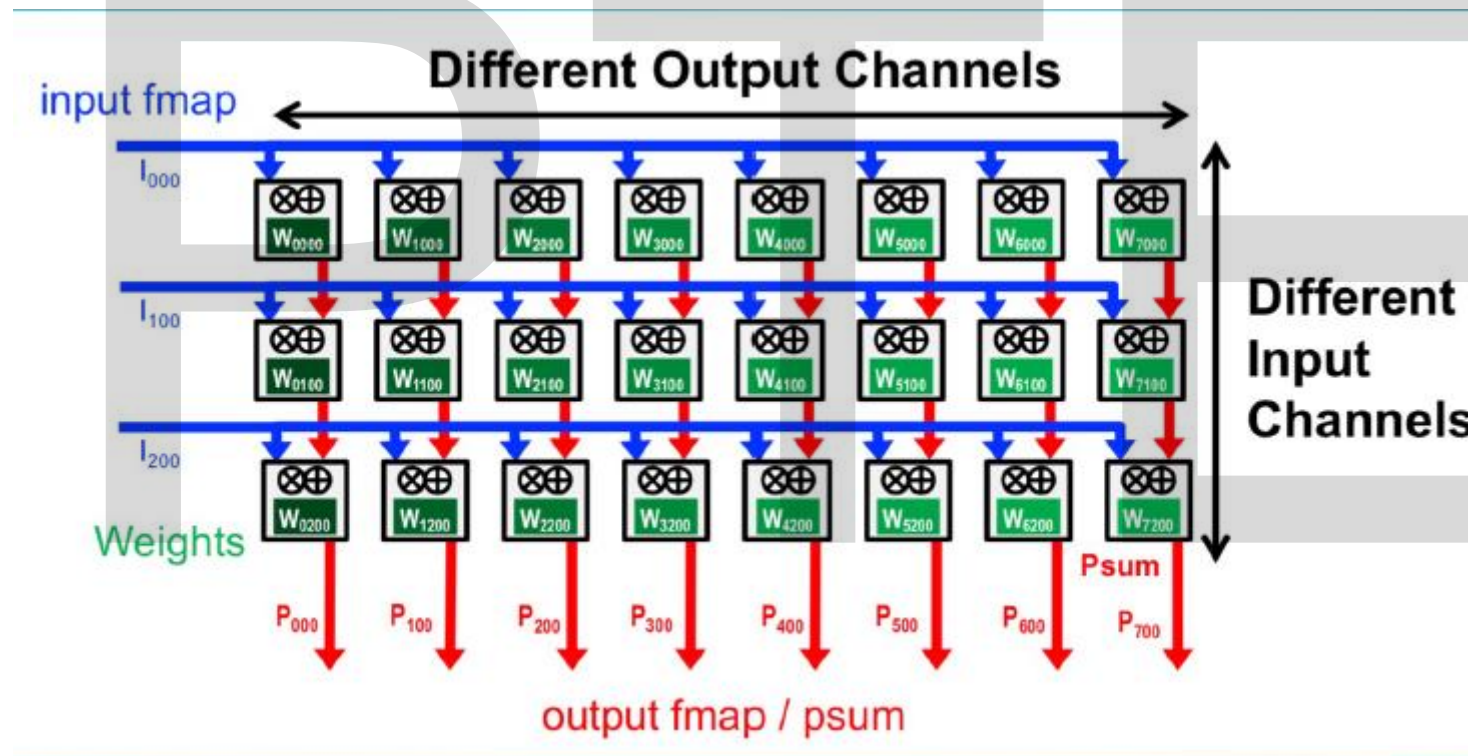


Image Source: Nvidia

<http://nvdla.org>

WS Example: NVDLA (simplified)

- NVIDIA Deep Learning Accelerator (NVDLA)



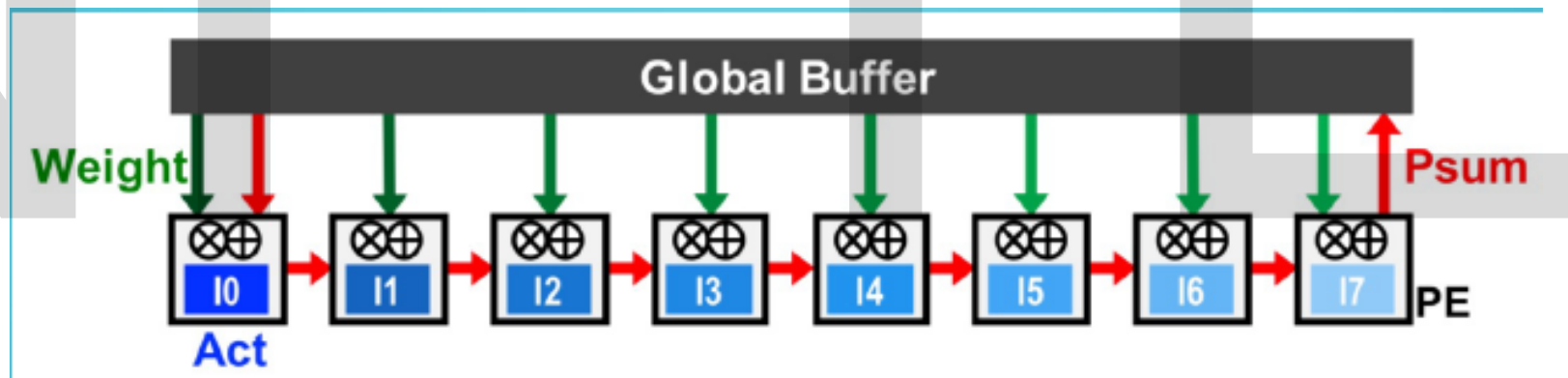
Different Types of Data Flow

- Output Stationary (OS)
- Weight Stationary (WS)
- **Input Stationary (IS)**

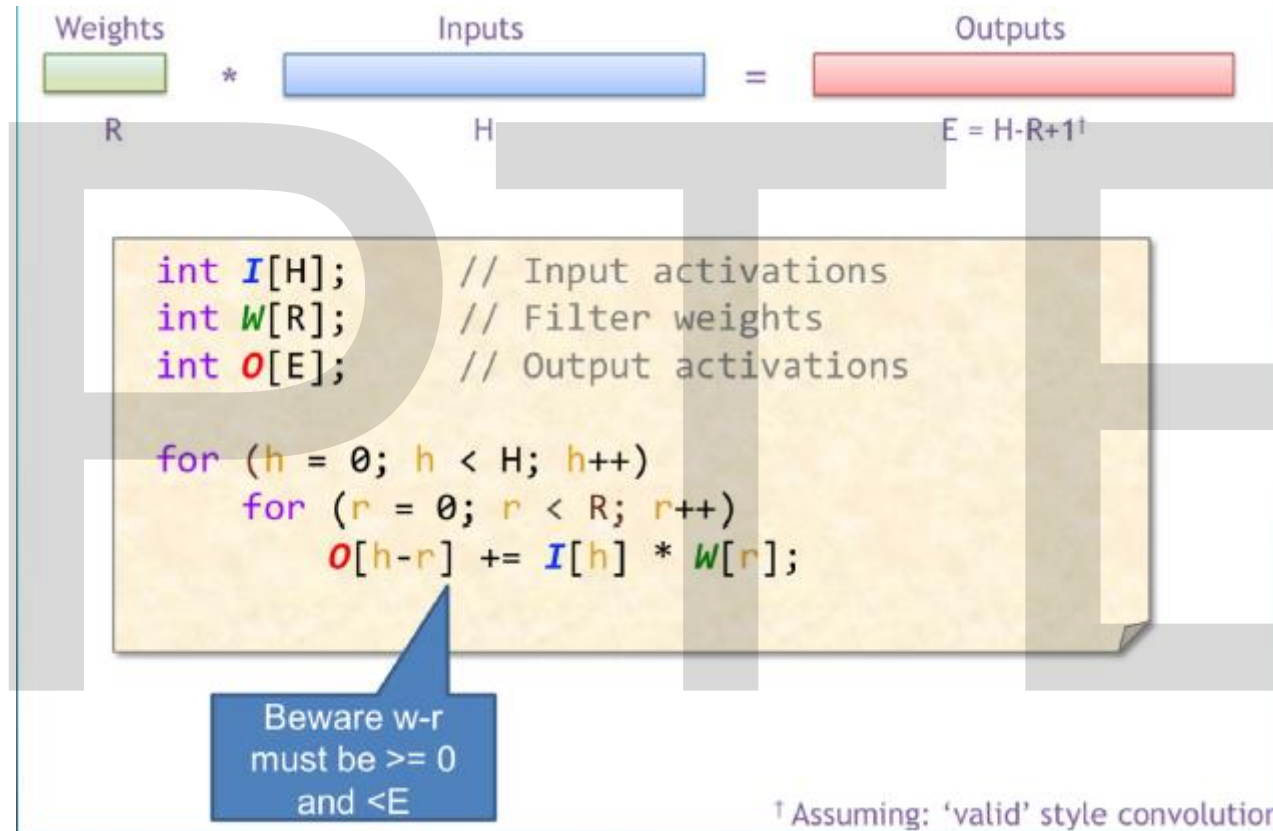
NPTEL

Input (IS) Data Flow

- **Minimize activation read energy consumption**
 - Input stays in PEs until they are no longer needed
 - Maximize convolutional and map reuse of activations
- **Unicast weights and accumulate psums spatially across the PE array**



IS Data Flow



Cost of different data flow

	OS	WS	IS	Min
MACs	$E \cdot R$	$E \cdot R$	$E \cdot R$	$E \cdot R$
Weight Reads	$E \cdot R$	R	$E \cdot R$	R
Input Reads	$E \cdot R$	$E \cdot R$	E	E
Output Reads	0	$E \cdot R$	$E \cdot R$	0
Output Writes	E	$E \cdot R$	$E \cdot R$	E

Assume: $W \sim E$



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)

